Lecture 11 · Wed 11/4/2026

CS160 Compilers — Fall 2026

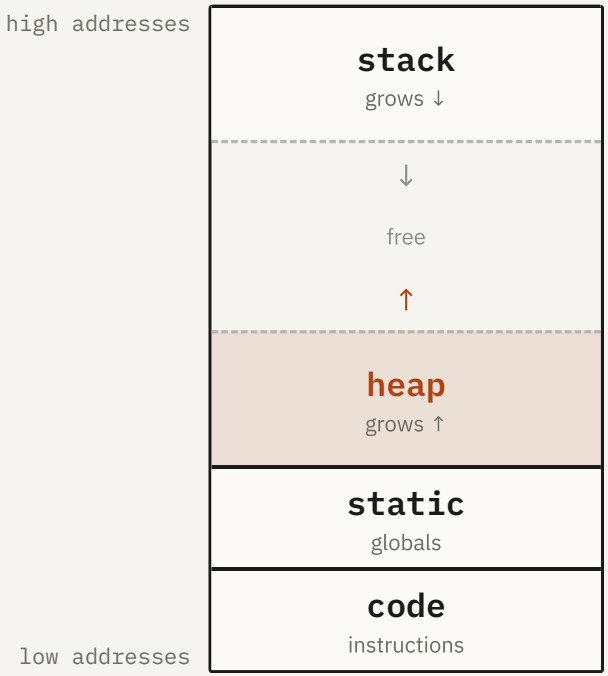
Runtime Organization.

Yu Feng

Where values live · 1/2

The four regions of memory.

Dragon Fig. 7.1



Code: the instructions. **Static**: things whose size and lifetime are the whole program — Lecture 8's `@x = global i32`.

Stack: one frame per activation; pushed on call, popped on return; sizes known at compile time (Lecture 8). Every value so far — an `int` or a `bool` in an `alloca` slot — lived here.

Heap: objects whose size is known only when the program runs, and whose lifetime is not tied to any call. PA4's lists live here. This is the new region.

Where values live · 2/2

Why a list cannot live on the stack.

mk.py · verified

```
def zeros(n: int) -> [int]:
    xs: [int] = ...          # a list of n zeros
    return xs                # xs must survive after zeros returns
```

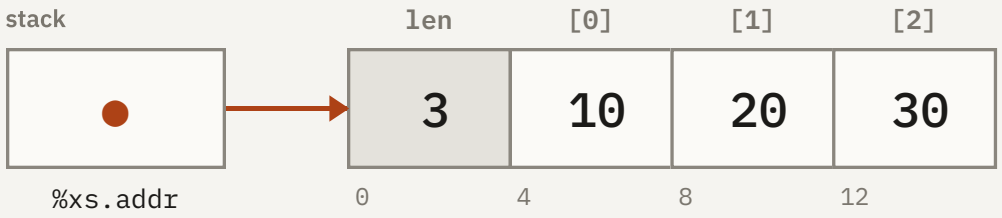
`alloca` dies with the frame; the list must not. `alloca i32` has a fixed size; the list's depends on `n`.

So the list goes on the heap, and the *variable* holds its address: `%xs.addr = alloca ptr`.

Nothing in the CPU manages the heap. The code that hands out heap memory is ours — the **runtime system**, the C file linked into every program your compiler produces. It is part of the compiler.

`ptr`: Lecture 3's pointer type: a value that *is* an address, 8 bytes.

The layout of a list: `xs: [int] = [10, 20, 30]`.



object layout
which bytes of an object hold what, counted from its first byte

header word
the 4 bytes at the front that describe the object rather than hold data; ours holds the length.

`xs[i]` starts at byte `4 * (i + 1)`. `len(xs)` is a load from byte 0. The object is `4 + 4 * len` bytes.

A list on the heap · 2/4

The runtime hands out heap memory.

runtime.c · verified

```
void *chocopy_alloc(int bytes) {          /* the heap, one function */
    void *p = calloc(bytes, 1);           /* zeroed */
    if (p == NULL) chocopy_error(3);      /* out of memory */
    return p;
}
```

NULL
the address zero, meaning *no object here*. LLVM IR spells it `null`; in ChocoPy it is what `None` is (slide 13).

`chocopy_error` gets messages: 1 division by zero, 2 index out of bounds, 3 out of memory — three of the manual's five runtime errors (Monday, slide 10). We never free; Lecture 15.

A list literal, as IR.

python3 cocoa11.py idx.py · verified · xs: [int] = [10, 20, 30] at top level

```
@xs = global ptr null                          ; the variable: a global holding an address
  %t1 = call ptr @chocopy_alloc(i32 16)        ; 4 + 3*4 bytes
  store i32 3, ptr %t1                         ; the header: length 3
  %t2 = getelementptr i32, ptr %t1, i64 1      ; the address of element 0: %t1 + 4
  store i32 10, ptr %t2
  %t3 = getelementptr i32, ptr %t1, i64 2
  store i32 20, ptr %t3
  %t4 = getelementptr i32, ptr %t1, i64 3
  store i32 30, ptr %t4
  store ptr %t1, ptr @xs                       ; remember the address
```

`getelementptr` (say “gep”)

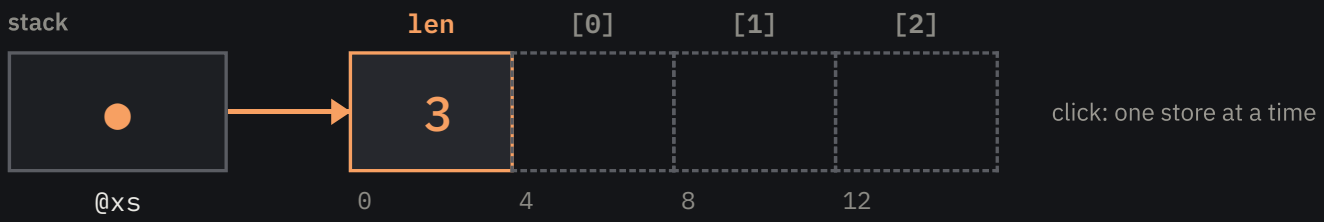
computes an address and does nothing else — here `%t1` plus one 4-byte step. Slide 8 is the full story.

Allocate, write the header, write each element, store the address in the variable. The variable holds a `ptr`, never the list.

The compiler's side.

cocoa11.py · verified

```
case ast.List(elts=es):                                          # [e1, e2, ...]
    obj = "%" + out.fresh()
    out.emit(f"{obj} = call ptr @chocopy_alloc(i32 {4 * (len(es) + 1)})")
    out.emit(f"store i32 {len(es)}, ptr {obj}")                  # header
    for k, elt in enumerate(es):
        v = compile_expr(elt, out); p = "%" + out.fresh()
        out.emit(f"{p} = getelementptr i32, ptr {obj}, i64 {k + 1}")
        out.emit(f"store i32 {v}, ptr {p}")
    return obj
```



click: one store at a time

`getelementptr`: address arithmetic with a type.

`gep T, ptr p, i64 k` is `p + k * sizeof(T)`. Nothing is loaded.

clang -O1 on `int third(int *p) { return p[3]; }` · verified · trimmed

```
%2 = getelementptr inbounds i32, ptr %0, i64 3      ; %0 + 12 bytes
%3 = load i32, ptr %2                               ; THIS reads memory
```

stride

the size of one step; the type says it — `i32` → 4 bytes, `i8` → 1

`i64 3`

the index is 64-bit because an address is

`inbounds`

clang's promise that the arithmetic stays inside one object; we check the index instead (slide 10).

Every element access is *two* instructions: `gep` to work out *where*, then `load` or `store` to *touch* it.

The rule for xs[i].

$$\frac{G, E, S \vdash xs : \ell, S_1, \text{None} \quad G, E, S_1 \vdash i : k, S_2, \text{None} \quad 0 \le k < \text{len}(\ell)}{G, E, S \vdash xs[i] : \text{elements}(\ell)[k], S_2, \text{None}}\ \text{E-Index}$$

$$\frac{G, E, S \vdash xs : \ell, S_1, \text{None} \quad G, E, S_1 \vdash i : k, S_2, \text{None} \quad k < 0 \lor k \ge \text{len}(\ell)}{G, E, S \vdash xs[i] : \textbf{error}, S_2, \text{None}}\ \text{E-IndexError}$$

bounds check
the third premise turned into instructions: compare the index against the length *before* touching memory; stop the program if it is out of range.

Monday's shape. The third premise is a check; when it fails the second rule fires and `error` is the outcome — `chocopy_error(2)`, exit 1.

In C there is no third premise, so there is no rule (slide 14).

xs[i] as IR, with the check.

from idx.py · verified · i = 2, prints 30

```
  %t5 = load ptr, ptr @xs
  %t6 = load i32, ptr @i
  %t8 = load i32, ptr %t5                    ; the length, from the header
  %t9 = icmp ult i32 %t6, %t8                ; unsigned:  0 <= i < len  in one compare
  br i1 %t9, label %inb7, label %oob7
oob7:
  call void @chocopy_error(i32 2)
  unreachable
inb7:
  %t10 = add i32 %t6, 1                      ; skip the header
  %t11 = getelementptr i32, ptr %t5, i32 %t10
  %t12 = load i32, ptr %t11
```

icmp ult

unsigned less-than, beside Lecture 3's `slt`: the same 32 bits, read as a non-negative number

why one compare is enough

a length is far below 2^{31}, so a negative `i`, read as unsigned, is larger than any length and fails the same compare that catches `i ≥ len`

unreachable

a terminator saying control never gets here: `chocopy_error` does not return, and every chunk must end in a jump, a `ret` or this.

One compare, one branch, one gep, one load. The two chunks are Lecture 3's `if`, with the error call as the else.

Reading and writing an element · 4/4

xs[i] = v and len.

cocoa11.py · compile_elem_ptr emits slide 10's check and returns the element's address

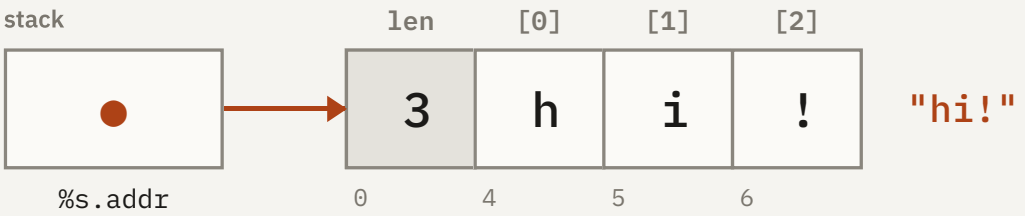
```
case ast.Assign(targets=[ast.Subscript(value=xs, slice=i)], value=v):
    p = compile_elem_ptr(xs, i, out)
    val = compile_expr(v, out)
    out.emit(f"store i32 {val}, ptr {p}")

case ast.Call(func=ast.Name(id="len"), args=[a]):
    xs = compile_expr(a, out); t = "%" + out.fresh()
    out.emit(f"{t} = load i32, ptr {xs}")          # the header
    return t
```



A read loads from the element's address; a write stores to it; the check is the same. `len` is a load from byte 0.

Strings: a list of bytes.



Same header; elements are `i8`, so the stride is 1: `gep i8, ptr %s, i64 k`.

`s[i]` builds a *new* one-character string object.

There is no `s[i] = c`: the checker rejects it. Immutability is a typing rule, not a runtime check.

`i8`: an 8-bit integer, one byte. LLVM names an integer type by its width: `i1`, `i8`, `i32`.

None is the null pointer.

`xs: [int] = None` is legal ChocoPy (`<None> ≤ₐ [int]`, Lecture 9 slide 4). `None` is the null pointer, so `len(None)` must be a runtime error, not a crash.

Wherever the checker says <None> is possible, the compiler emits `icmp eq ptr %xs, null` before the header load and calls `chocopy_error` — the manual's fourth runtime error, *operation on None*.

The same bug in C.

offby1.py · the loop runs one too far

```
def total(xs: [int]) -> int:
    s: int = 0
    i: int = 0
    while i <= len(xs):
        s = s + xs[i]
        i = i + 1
    return s
```

verified · pre-rendered

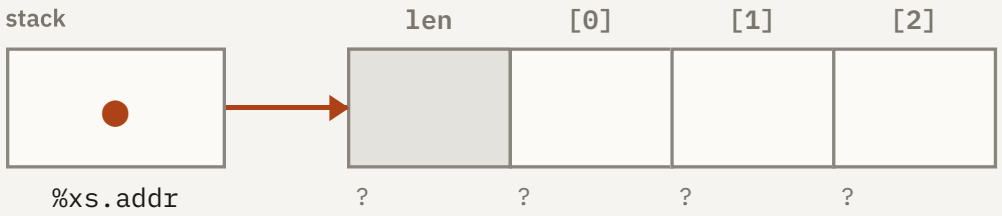
```
ChocoPy
runtime error 2: index out of bounds  exit 1

offby1.c, -O0
205116740
offby1.c, -O2
-1967765880
```



ChocoPy: `E-IndexError` fires. C: no rule, so each compiler reads whatever sits past the array, and both outputs are *correct* — Monday's undefined behavior. The price of the rule is slide 10's compare and branch, on every access.

Exercise #1: draw the heap object for `[1, 2, 3]`.



Header, three cells, byte offsets. Which byte does `xs[2]` start at? Then the gep and `load` that read it, given %xs holds the address.

Exercise #2: write the IR for `xs[i] = v`, with the check.

`length load` `icmp ult` `br` `error chunk` `gep` `store`

`%xs` (ptr), `%i` and `%v` (i32) already loaded. Length load, `icmp ult`, `br`, error chunk, gep, `store`. Labels `inb1` / `oob1`.

Midterm 2: Lectures 7–11, pencil and paper.

Parsing II	L7	FIRST/FOLLOW and an LL(1) table; a shift-reduce trace; an LR(0) collection and its SLR table; reading a conflict.
Functions	L8	the stack at the deepest point of a recursion; what a caller must save.
Types	L9	a typing derivation; which rule fails.
Semantics	L10	a big-step derivation; evaluation order; is a rewrite valid.
Runtime	L11	a heap object with offsets; bounds-checked IR by hand.

Midterm 2: two sample questions.

(a) `def f(a: int, b: int) -> int: return g(b) + a`. Draw `f`'s frame while `g` runs: parameters, return address, and where `a` is kept so it survives the call.

(b) `ys: [int] = zeros(4)` then `ys[3] = 7`. Draw the heap object after the store; which byte offset changed?

(a) `a` in a callee-saved register or spilled to `%a.addr`. (b) 16: header at 0, `[3]` at `4 * 4`.

PA4; after the midterm.

PA4 (due Fri 11/13): lists and strings — slides 6–11 are the design; `chocopy_alloc` is in the runtime you are given; the None check of slide 13 is yours. Thursday's section is the PA4 lab.

No lecture Wed 11/11. Then Lecture 12: the IR as a data structure, and the form that makes optimization possible.